

Arduino Based Microcontroller (CSE 4820)

CHAPTER 3 | MATHEMATICAL OPERATIONS



Program: B. TECH., 7th Semester

SIKSHA 'O' ANUSHANDHAN (Deemed to be) UNIVERSITY

Chapter 3 |

- **Arithmetic**
- **Increment/decrement**
- **Remainder / modulus**
- **Absolute value**
- **Constrain**
- **Minimum / maximum**
- **Power**
- **Square root**
- **Floor / ceil / round**
- **Trigonometric functions**
- **Random numbers**
- **Setting / reading bits**
- **Shifting bits**
- **High / low bytes**
- **Forming int / long from bytes**

Chapter 3 | Introduction

- Mathematical operations are an essential part of almost every [Arduino sketch](#).
- They are used [to manipulate](#) and [process the values](#) stored in variables.
- This chapter provides an [overview of](#) the most common [mathematical operations](#) used in Arduino programming.
- The chapter introduces some Arduino programming idioms and techniques that may be encountered in Arduino sketches.

3.1 | Arithmetic Operators

Problem

- ❑ Perform basic mathematical operations on values in an Arduino sketch.
- ❑ Control the order of mathematical operations.
- ❑ Handle different variable types correctly.

Operator	Operation	Example
+	Addition	$1 + 2 = 3$
-	Subtraction	$3 - 2 = 1$
*	Multiplication	$3 * 2 = 6$
/	Division	$3 / 2 = 1$

Solution — Basic Arithmetic

```
int myValue;  
  
myValue = 1 + 2; // Addition  
myValue = 3 - 2; // Subtraction  
myValue = 3 * 2; // Multiplication  
myValue = 3 / 2; // Division → result is 1
```

Discussion

- Addition (+), subtraction (-), and multiplication (*) work as expected for integers.
- Integer division (/) truncates the fractional part. Therefore $3/2$ is 1.

3.1 | Adding, Subtracting, Multiplying and Dividing Cont.....

Discussion — Arithmetic Operations

Example:1

```
int a = 7 / 2;      //Ans: 3
```

Example:2

```
float b = 7.0 / 2;  // Ans: 3.5
```

Integer Division

- When both operands are integers, the fractional part is discarded.

Example: $7 / 2 = 3$.

- If one operand is floating-point, a fractional result can be obtained:

Example: $7.0 / 2 = 3.5$.

Remember: If you need a fractional result, use an appropriate floating-point variable/type.

3.1 — Operator Precedence

```
int value = 1 + 2 * 3 + 4;    // Ans: 11
int value2 = 1 + (2 * 3) + 4; // Ans: 11
int value3 = ((1 + 2) * 3) + 4; // Ans: 13
```

➤ **Multiplication and division** have **higher precedence** than addition and subtraction.

$1 + 2 * 3 + 4 = 11$. Therefore, multiplication is performed first: $1 + (2 \times 3) + 4 = 11$

➤ Parentheses make the intended order clear. $((1 + 2) * 3) + 4 = 13$.

Calculation order: Step 1: $(1 + 2) = 3$

Step 2: $3 \times 3 = 9$

Step 3: $9 + 4 = 13$

3.1 | Overflow: Example

Problem:

Even when the destination variable has the correct data type, overflow can still occur during the calculation.

```
// 60 sec/min × 60 min/hour × 24 hours/day
```

```
long seconds_per_day = 60 * 60 * 24;
```

What do we expect?

Mathematical result: $60 \times 60 \times 24 = 86,400$

86,400 can easily fit in a long.

So, we may expect: result as **// seconds_per_day = 86400**

3.1 | Overflow: Example Cont.....

Why Does Overflow Occur?

- The calculation is performed before the result is assigned to the long variable.
- All the numbers in this expression are integer constants: $60 * 60 * 24$
- So the compiler performs the calculation as integer arithmetic first. $60 \times 60 \times 24 = 86,400$
- But the Arduino integer range is: -32,768 to +32,767
- Therefore, 86,400 is too large for an integer.
- The calculation overflows before its result is placed into: **long seconds_per_day**
- The Arduino stores the resulting value as: $86,400 - (32,768 \times 2) = 20,864$

❓ 3.1 | Overflow: Example Cont.....

Correct Overflow Handling

- Appending **L** to the first appropriate value makes the calculation use **long** arithmetic.
- The location of **L** matters when parentheses are present because inner parentheses are evaluated first.

Example:

```
long seconds_per_day = 60L * 60 * 24;
```

Why does this work?

60L tells the compiler to treat this value as a long. Therefore, the arithmetic expression is evaluated using the long type.

60L × 60 × 24
↓
86,400
↓
Stored in long
↓
Correct result = 86,400

3.1 | Overflow + Parentheses

// With parentheses:

```
long a = 1L + 60 * (60 * 24);    // overflows
```

```
long b = 1 + 60 * (60L * 24);    // correct
```

Important: Parentheses Can [Change the Result](#) as the innermost parentheses are evaluated first.

$1L + 60 * (60 * 24)$ can still overflow because the inner expression is evaluated first.

Correct example: $1 + 60 * (60L * 24)$.

3.1 | Floating-Point Precision

- Do not assume that floating-point calculations will always produce an exact decimal representation.

Example:-- $36.3/3$ (Mathematically: $36.3 \div 3 = 12.1000000000$)

But the Arduino output can be: 12.0999994277

```
Serial.println(36.3/3,10); // 12.0999994277
```

Reason: Floating-point numbers **cannot** represent **every decimal value exactly** in binary. Therefore, a very small representation error can appear during calculation.

3.2 | Increment & Decrement

Problem:

You want to increase or decrease the value of a variable in your Arduino sketch.

- Increase a value → Increment
- Decrease a value → Decrement
- These are very common operations in programming.

Long form: $x = x + 1$ / $x = x - 1$.

Compound form: $x += 1$ / $x -= 1$.

Short form: $x++$ / $x--$.

3.2 | Increment & Decrement Cont

Basic Method (Long form)

```
int myValue = 0;  
myValue = myValue+1; // add 1  
myValue = myValue-1; // subtract 1  
myValue = myValue+5; // add 5
```

Explanation

- myValue = myValue + 1 increases myValue by 1.
- myValue = myValue - 1 decreases myValue by 1.
- Similarly, myValue = myValue + 5 increases it by 5.

3.2 | Increment & Decrement Cont

Compound Assignment Operators

Instead of writing the long form, we can use compound assignment operators. It makes the code shorter and easier to write.

`myValue += 1; // same as myValue = myValue + 1`

`myValue -= 1; // same as myValue = myValue - 1`

`myValue += 5; // same as myValue = myValue + 5`

3.2 | Increment & Decrement Cont

Increment and Decrement Operators (short form)

- When the change is exactly 1, Arduino/C++ provides special operators.
- The Two Forms: (Pre and Post) ++ and -- operators can appear

After the variable → Post-increment/decrement

myValue++; // increment by 1

myValue--; // decrement by 1

Before the variable → Pre-increment/decrement

++myValue;

--myValue;

3.2 | Increment & Decrement Cont

Example:

```
int myVal = 1;
```

```
Serial.println(myVal++); // prints 1
Serial.println(myVal);   // prints 2
Serial.println(++myVal); // prints 3
Serial.println(myVal);   // prints 3
```

Step	Statement	What happens?	Printed value	myVal after statement
1	Serial.println(myVal++);	Post-increment: print first, then increase	1	2
2	Serial.println(myVal);	Print the current value	2	2
3	Serial.println(++myVal);	Pre-increment: increase first, then print	3	3
4	Serial.println(myVal);	Print the current value	3	3

3.3 | Finding the Remainder After Dividing Two Values

- To find the remainder after dividing two integer values modulus operator % is used.
- Arduino/C++ provides the modulus operator % for this purpose.

% returns the remainder.

```
int myValue0 = 20 % 10; // get the modulus(remainder) of 20 divided by 10
```

```
int myValue1 = 21 % 10; // get the modulus(remainder) of 21 divided by 10
```

Expression	Division	Remainder
20 % 10	20 ÷ 10	0
21 % 10	21 ÷ 10	1

3.3 | Finding the Remainder After Dividing Two Values Cont...

Discussion

Example 1-- Write a program to determine whether a number is a multiple of 10?

Answer:

Rule: `myValue % 10 == 0`

If the remainder is 0, the number is a multiple of 10.

Program

```
for (int myValue = 0; myValue <= 100; myValue += 5)
{
    if (myValue % 10 == 0)
    {
        Serial.println("The value is a multiple of 10");
    }
}
```

3.3 | Finding the Remainder After Dividing Two Values Cont...

Discussion

Example 2 -- Write a program to check if a value is odd or even?

Answer: Rule: An even number is exactly divisible by 2: $\text{number \% 2} == 0$

An odd number leaves a remainder: $\text{number \% 2} != 0$

Program

```
for (int myValue = 0; myValue <= 10; myValue++)  
{  
    if (myValue % 2 == 0)  
    {  
        Serial.println("The value is even");  
    }  
    else  
    {  
        Serial.println("The value is odd");  
    }  
}
```

3.4 | Determining the Absolute Value : abs()

- **abs(x)** gives the absolute value for integer/long values.
- Useful for comparing sensor readings regardless of sign.

Discussion

Example: **Write a program to compute the absolute value of the difference between readings on two analog input ports.**

PROGRAM

```
int x = analogRead(A0);  
int y = analogRead(A1);  
  
if (abs(x-y) > 10)  
{  
    Serial.println("The analog values differ by more than 10");  
}
```

3.5 | Constraining a Number to a Range : constrain()

- To ensure that a value always remains within a lower and upper limit **constrain()** is used.
- **constrain(x,min,max)** returns a value inside the inclusive range.

Dicussion

`int myConstrainedValue = constrain(myValue, 100, 200);` → $100 \leq \text{myConstrainedValue} \leq 200$

If `myValue < 100` → result becomes 100

If $100 \leq \text{myValue} \leq 200$ → result remains `myValue`

If `myValue > 200` → result becomes 200

myValue	constrain(myValue,100,200)
99	100
100	100
150	150
200	200
201	200

3.6 | Finding Minimum or Maximum

To find the minimum or maximum of two or more values min() and max() are used respectively.

min(x,y) // returns the smaller value

max(x.v) // returns the larger value

```
int myValue = analogRead(A0);  
  
int myMinValue = min(myValue, 200);  
  
int myMaxValue = max(myValue, 100);
```

Means

myMinValue → smaller of myValue and 200
myMaxValue → larger of myValue and 100

myValue	min(myValue,200)	max(myValue,100)
99	99	100
100	100	100
150	150	150
200	200	200
201	200	201

3.6 | constrain() vs min() vs max()

Function	Purpose	Example	Effect
abs()	Find absolute value	abs(-20)	20
constrain()	Keep value within a range	constrain(x,100,200)	100–200
min()	Find smaller value / limit upper bound	min(x,200)	Maximum output = 200
max()	Find larger value / limit lower bound	max(x,100)	Minimum output = 100

3.6 Finding Minimum of More Than Two Values

min() normally compares two values.

For more values, we can **cascade/nest** the function.

Example 1

```
int myMinValue =  
    min(analogRead(0),  
        min(analogRead(1), analogRead(2)));
```

Discussion

Example 1

How does it work?

`min(analogRead(1), analogRead(2))`
finds the smaller value of ports 1 and 2.
Then `min(port 0, result)` finds the smaller value
between port 0 and that result.

Final Result

`myMinValue` = smallest of the three analog readings.

3.6 Finding Minimum of More Than Two Values Cont.....

Example 2

```
int myMaxValue =  
    max(analogRead(0),  
        max(analogRead(1),  
            max(analogRead(2),  
                analogRead(3))));
```

Discussion

Example 2 How does it work?

Step 1:

max(analogRead(2), analogRead(3))

↓

Step 2:

max(analogRead(1), result)

↓

Step 3:

max(analogRead(0), result)

↓

Final maximum

Important-When nesting min() or max():

Be very careful with the placement of parentheses. Incorrect parentheses can change the calculation.

3.7 | Raising a Number to a Power : pow()

- To raise a number to a power pow() is used.
- pow(x,y) raises x to the power y.
- It Accepts integer or floating-point values; returns floating point.

Example 1

```
int myValue = pow(3, 2); // result is : myValue = 9 (32 = 9)
```

Example 2

```
Serial.println(pow(3,2)); // result is : 9.00
```

Example 3

```
int z = pow(3,2);  
Serial.println(z); // result is : 9
```

The mathematical result is the same, but the data type used for display is different.

3.8 | Taking the Square Root `sqrt()`

- To calculate the square root of a number `sqrt(x)` returns the square root.

```
Serial.println(sqrt(9)); // this prints 3.00
```

3.9 | Rounding Floating-Point Numbers Up and Down

`floor()`, `ceil()`, `round()`

`floor(x)`: largest integer not greater than x.

`ceil(x)`: smallest integer not less than x.

`round(x)`: nearest integer.

Discussion:

```
Serial.println( floor(1) ); // this prints 1.00
```

```
Serial.println( floor(1.1) ); // this prints 1.00
```

```
Serial.println( floor(0) ); // this prints 0.00
```

```
Serial.println( floor(0.1) ); // this prints 0.00
```

```
Serial.println( floor(-1) ); // this prints -1.00
```

```
Serial.println( floor(-1.1) ); // this prints -2.00
```

3.9 | Rounding Floating-Point Numbers Up and Down

`floor()`, `ceil()`, `round()` Cont....

Discussion:

`Serial.println( ceil(1) );` // this prints 1.00

`Serial.println( ceil(1.1) );` // this prints 2.00

`Serial.println( ceil(0) );` // this prints 0.00

`Serial.println( ceil(.1) );` // this prints 1.00

`Serial.println( ceil(-1) );` // this prints -1.00

`Serial.println( ceil(-1.1) );` // this prints -1.00

`round(1.2)` → 1

`round(5.6)` → 6

For values exactly halfway between integers, Arduino/C++ `round()` rounds away from zero:

`round(1.5)` → 2

`round(-1.5)` → -2

3.10 | Using Trigonometric Functions

Arduino provides three common trigonometric functions:

`sin(x)` // sine

`cos(x)` // cosine

`tan(x)` // tangent

The trigonometric functions expect the angle in **radians**, not degrees.

```
float deg = 30;
```

```
float rad = deg * PI / 180; //Convert degrees to radians:
```

```
//  $30^\circ \times \pi / 180 = 0.5235988$  radians
```

```
Serial.println(sin(rad), 5); //  $\sin(30^\circ) = 0.50000$ 
```

```
Serial.println(cos(rad), 5); //  $\cos(30^\circ) = 0.86603$ 
```

3.11 | Generating Random Numbers : random()

➤ Arduino provides the random() function to generate pseudo-random integer values.

There are two commonly used forms:

1. random(max)

long r = random(100); //This generates a number from: 0 to 99

2. random(min, max)

Suppose we want a random number from **50 to 99**

long r = random(50, 100); // The result will be: $50 \leq r < 100$

What is randomSeed()?

randomSeed(1234); //the sequence will start from the same point each time.

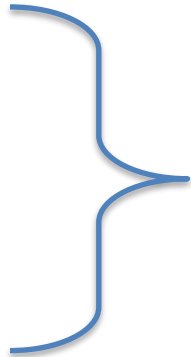
randomSeed() sets the starting point (seed) of the pseudo-random number sequence.

Arduino's random() does not generate truly random numbers. It generates a sequence that appears random.

3.11 | Generating Random Numbers random()

Different forms of random number generation available on Arduino:

```
// Random
// demonstrates generating random numbers
int randNumber;
void setup()
{
  Serial.begin(9600);
  while(!Serial);
  // Print random numbers with no seed value
  Serial.println("Print 20 random numbers between 0 and 9");
  for(int i=0; i < 20; i++)
  {
    randNumber = random(10);
    Serial.print(randNumber);
    Serial.print(" ");
  }
```



Explanation: Generate random numbers from 0 to 9

- random(10) generates a random number from 0 to 9.
- The for loop executes 20 times.
- Therefore, 20 random numbers are displayed.
- Serial.print() displays the numbers on the Serial Monitor.

3.11 | Generating Random Numbers random() Cont...

```
Serial.println();
```

```
Serial.println("Print 20 random numbers between 2 and 9");
```

```
for(int i=0; i < 20; i++)  
{  
  randomNumber = random(2,10);  
  Serial.print(randomNumber);  
  Serial.print(" ");  
}
```

```
// Print random numbers with the same seed value each time
```

```
randomSeed(1234);
```

```
Serial.println();
```

```
Serial.println("Print 20 random numbers between 0 and 9 after constant seed  
");
```

```
for(int i=0; i < 20; i++)  
{  
  randomNumber = random(10);  
  Serial.print(randomNumber);  
  Serial.print(" ");  
}
```

Explanation: Generate random numbers from 2 to 9

- random(2,10) specifies the lower and upper limits.
- The lower limit 2 is included.
- The upper limit 10 is excluded.

Explanation: Generate a repeatable sequence using a fixed seed

- randomSeed(1234) sets the starting point of the pseudorandom sequence.
- The seed value is fixed at 1234.
- Therefore, whenever the sketch starts with the same seed, the same sequence of numbers is generated.

3.12 Setting and Reading Bits

A bit is a single binary digit that can have only two states:

0 → OFF

1 → ON

A single variable can contain **many bits**, so we can use different bits to store several independent ON/OFF conditions.

For example, an 8-bit variable:

Bit position: 7 6 5 4 3 2 1 0

Value(x): 1 0 1 0 0 1 0 1

Each bit can represent a separate condition.

1. bitSet(x, bitPosition)

Sets a particular bit to 1.

`bitSet(x, 3);` //Set bit number 3 of x to 1.

2. bitClear(x, bitPosition)

Sets a particular bit to 0.

`bitClear(x, 3);` //Clear bit number 3.

3. `bitRead(x, bitPosition)`

Reads the value of a particular bit.

`int b = bitRead(x, 3);` //It returns either:0 or 1

For example, if : x = 00001000 then:

`bitRead(x, 3)` returns : 1 because bit 3 is 1.

4. `bitWrite(x, bitPosition, value)`

Used to write either 0 or 1 to a particular bit.

`bitWrite(x, 3, 1);` //Write 1 into bit 3.

`bitWrite(x, 3, 0);` //Write 0 into bit 3.

So `bitWrite()` can be thought of as a combination of set and clear.

5. `bit(bitPosition)`

This one is slightly different.

`bit(3)` // returns the numerical value represented by bit position 3:

`bit(0) = 1`

`bit(1) = 0`

Shifting Bits – Left and Right Shift

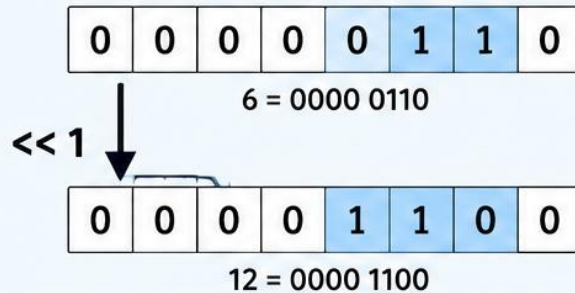
Shifting means moving the binary bits (0s and 1s) to the left or right.

Left Shift (<<)

- Moves the bits to the left.
- Fills 0 on the right side.
- Each left shift by 1 is equivalent to multiplying by 2.

Example: 6 << 1

6 (decimal) = 0000 0110 (8-bit representation)



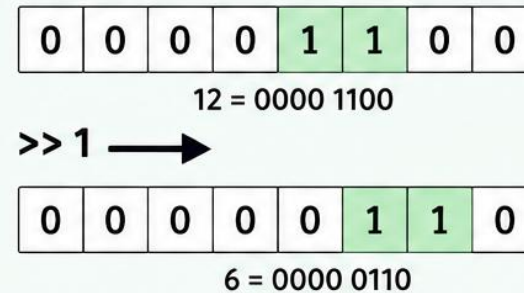
So, $6 \ll 1 = 12$ (6×2)

Right Shift (>>)

- Moves the bits to the right.
- Fills 0 on the left side.
- Each right shift by 1 is equivalent to dividing by 2 (for positive numbers).

Example: 12 >> 1

12 (decimal) = 0000 1100 (8-bit representation)



So, $12 \gg 1 = 6$ ($12 \div 2$)

More Examples

Operation	Result (Decimal)	Meaning
$6 \ll 1$	12	6×2
$6 \ll 2$	24	6×4
$6 \ll 3$	48	6×8

$12 \gg 1$	6	$12 \div 2$
$12 \gg 2$	3	$12 \div 4$
$12 \gg 3$	1	$12 \div 8$

Key Takeaways

- << : shift left → multiply by 2
- >> : shift right → divide by 2
- A left shift fills 0 on the right.
- A right shift fills 0 on the left.
- Useful for fast multiplication/division, working with bits, flags, and registers (in Arduino and embedded systems).

Code Example (Arduino/C/C++)

```
int x = 6;           // 0110
x = x << 1;          // 1100 = 12
Serial.println(x);    // 12
```

```
x = x >> 2;          // 0011 = 3
Serial.println(x);    // 3
```

Shifting bits is a simple and efficient way to work with binary numbers!

Extracting High & Low Bytes from int / long

An int has 2 bytes (16 bits) and a long has 4 bytes (32 bits).

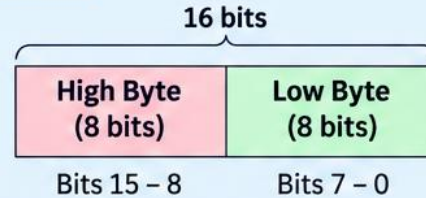
We can extract the high byte (most significant) and low byte (least significant) using bitwise operations.

1. int (2 bytes = 16 bits)

An int is 16 bits (2 bytes).

It can be divided into:

- High byte → upper 8 bits
- Low byte → lower 8 bits



Example: int x = 0x1234; (4660 in decimal)

Binary representation:

0001 0010 0011 0100

High Byte	Low Byte
0001 0010	0011 0100
(0x12)	(0x34)

Code:

```
int x = 0x1234;
byte high = (x >> 8) & 0xFF;
byte low = x & 0xFF;
Serial.println(high, HEX); // 12
Serial.println(low, HEX); // 34
```

Result:

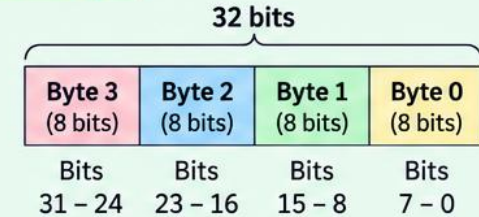
High byte = 0x12 (18 in decimal)
Low byte = 0x34 (52 in decimal)

2. long (4 bytes = 32 bits)

A long is 32 bits (4 bytes).

It can be divided into:

- Byte 3 (highest) → bits 31 – 24
- Byte 2 → bits 23 – 16
- Byte 1 → bits 15 – 8
- Byte 0 (lowest) → bits 7 – 0



Example: long x = 0x12345678; (305419896 in decimal)

Binary representation:

0001 0010 0011 0100 0101 0110 0111 1000

Byte 3	Byte 2	Byte 1	Byte 0
0001 0010	0011 0100	0101 0110	0111 1000
(0x12)	(0x34)	(0x56)	(0x78)

Code:

```
long x = 0x12345678;
byte b3 = (x >> 24) & 0xFF; // 0x12
byte b2 = (x >> 16) & 0xFF; // 0x34
byte b1 = (x >> 8) & 0xFF; // 0x56
byte b0 = x & 0xFF; // 0x78
```

Result:

Byte 3 = 0x12 (18)
Byte 2 = 0x34 (52)
Byte 1 = 0x56 (86)
Byte 0 = 0x78 (120)

Arduino Helper Functions

In Arduino, you can also use built-in functions:

highByte(x) // returns the high byte of int or long
lowByte(x) // returns the low byte of int or long

Example (Arduino)

```
int x = 0x1234;
Serial.println(highByte(x), HEX); // 12
Serial.println(lowByte(x), HEX); // 34
```

Key Takeaways

- High byte contains the most significant bits.
- Low byte contains the least significant bits.
- Use bitwise shift (>>) and mask (& 0xFF) to extract.
- Arduino provides **highByte()** and **lowByte()** for easy use.
- Useful when working with communication, sensors, and binary data (e.g., sending multi-byte values).

3.15 Forming an int or long from High and Low Bytes

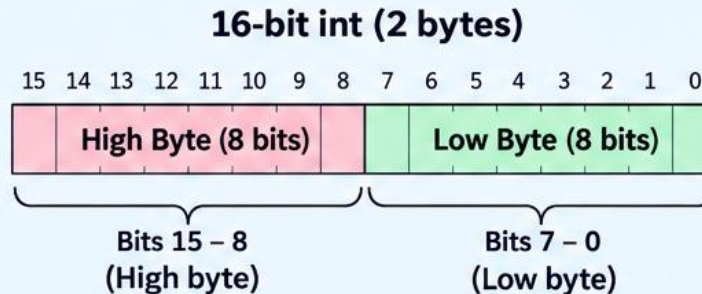
- Sometimes a 16-bit int or 32-bit long value is available as individual bytes or words.
- When an integer is received as separate bytes through a serial communication link, these bytes must be combined to reconstruct the original value.
- The objective is to form a 16-bit int from two individual bytes and a 32-bit long from two 16-bit words. This operation is the inverse of separating a value into its high and low bytes.
- Arduino provides the `word()` function to combine two bytes into a single integer.

Forming an int from High and Low Bytes

Combine the high byte and low byte to get a 16-bit int value (2 bytes) using bitwise operations.

1. What is an int?

- In Arduino (AVR), an int is 16 bits (2 bytes).
- It consists of:
 - High byte** → upper 8 bits (bits 15 – 8)
 - Low byte** → lower 8 bits (bits 7 – 0)
- We can form the original int value by placing the high byte in the upper 8 bits and the low byte in the lower 8 bits.



2. Formula

```
int x = ((int)high << 8) | low;
```

- `(int)high` → ensures high is treated as 16-bit value before shifting.
- `<< 8` → moves the high byte to the upper 8 bits.
- `|` → bitwise OR to combine with the low byte.
- `low` → lower 8 bits (no shift needed).

3. Example Values

Let the high byte be 0x12 and the low byte be 0x34.

Name	Hex	Binary (8 bits)	Decimal
High byte	0x12	0001 0010	18
Low byte	0x34	0011 0100	52

4. Step 1 – Shift the High Byte

Shift the high byte left by 8 bits to place it in the upper byte.

```
(int)high << 8 → 0x12 << 8 = 0x1200
```

Binary after shifting:

0001	0010	0000	0000
------	------	------	------

0x1200 (4608 in decimal)

5. Step 2 – Combine with Low Byte

Use bitwise OR to add the low byte.

```
0x1200 | 0x0034 = 0x1234
```

Binary after OR:

0001	0010	0011	0100
------	------	------	------

0x1234 (4660 in decimal)

6. Complete Code Example (Arduino/C)

```
byte high = 0x12; // 18
byte low = 0x34; // 52

int x = ((int)high << 8) | low;

Serial.print("High byte: 0x");
Serial.println(high, HEX);
Serial.print("Low byte: 0x");
Serial.println(low, HEX);
Serial.print("Combined int: 0x");
Serial.println(x, HEX);
Serial.print("Combined int (dec): ");
Serial.println(x);
```

Output (Serial Monitor)

High byte: 0x12
Low byte: 0x34
Combined int: 0x1234
Combined int (dec): 4660

7. Another Example

Let high = 0xAB and low = 0xCD.

Name	Hex	Binary (8 bits)	Decimal
High byte	0xAB	1010 1011	171
Low byte	0xCD	1100 1101	205

```
(int)high << 8 → 0xAB << 8 = 0xAB00
```

```
0xAB00 | 0x00CD = 0xABCD
```

So, x = 0xABCD (43981 in decimal)

8. Key Takeaways

- An `int` is 16 bits (2 bytes).
- High byte contains the most significant 8 bits.
- Low byte contains the least significant 8 bits.
- Shift the high byte left by 8 bits and use bitwise OR to combine.
- Always cast the high byte to `int` before shifting.
- This method is useful when receiving data in two separate bytes (e.g., from sensors, communication, etc.).

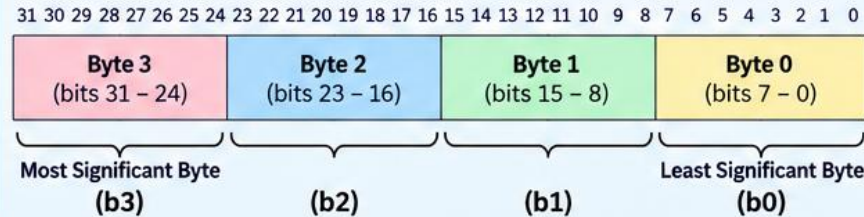
Forming a long from High and Low Bytes

Combine four bytes (b3, b2, b1, b0) to get a 32-bit long value (4 bytes) using bitwise operations.

1. What is a long?

- In Arduino (AVR), a long is 32 bits (4 bytes).
- It consists of 4 bytes:
 - Byte 3 (b3)** → bits 31 – 24 (most significant)
 - Byte 2 (b2)** → bits 23 – 16
 - Byte 1 (b1)** → bits 15 – 8
 - Byte 0 (b0)** → bits 7 – 0 (least significant)
- We can form the original long value by shifting each byte to its correct position and combining them using bitwise OR (|).

2. Structure of a 32-bit long



3. Formula

```
long x = ((long)b3 << 24) |  
          ((long)b2 << 16) |  
          ((long)b1 << 8) |  
          ((long)b0);
```

- Each byte is shifted to its correct position (24, 16, 8, 0 bits).
- Bitwise OR (|) is used to combine the bytes.
- (long) cast ensures the shift is done using 32 bits.

4. Example Values

Let the four bytes be:

b3 = 0x12, b2 = 0x34, b1 = 0x56, b0 = 0x78

Name	Hex	Binary (8 bits)	Decimal
b3 (Byte 3)	0x12	0001 0010	18
b2 (Byte 2)	0x34	0011 0100	52
b1 (Byte 1)	0x56	0101 0110	86
b0 (Byte 0)	0x78	0111 1000	120

5. Step-by-Step Formation

- Shift b3 left by 24 bits**
 $(\text{long})b3 \ll 24 \rightarrow 0x12 \ll 24 = 0x12000000$
(301989888 in decimal)
- Shift b2 left by 16 bits**
 $(\text{long})b2 \ll 16 \rightarrow 0x34 \ll 16 = 0x00340000$
(3407872 in decimal)
- Shift b1 left by 8 bits**
 $(\text{long})b1 \ll 8 \rightarrow 0x56 \ll 8 = 0x00005600$
(22016 in decimal)
- b0 remains as it is**
 $(\text{long})b0 \rightarrow 0x78 = 0x00000078$ (120 in decimal)

6. Combine Using Bitwise OR

OR all the shifted values:

```
0x12000000 | 0x00340000 |  
0x00005600 | 0x00000078  
= 0x12345678
```

Binary representation:

0001 0010	0011 0100	0101 0110	0111 1000
32 bits			
0x12345678 (305419896 in decimal)			

7. Complete Code Example (Arduino/C)

```
byte b3 = 0x12; // 18  
byte b2 = 0x34; // 52  
byte b1 = 0x56; // 86  
byte b0 = 0x78; // 120  
  
long x = ((long)b3 << 24) |  
          ((long)b2 << 16) |  
          ((long)b1 << 8) |  
          ((long)b0);  
  
Serial.print("Combined long: 0x");  
Serial.println(x, HEX); // 12345678  
Serial.print("Combined long (dec): ");  
Serial.println(x); // 305419896
```

8. Output (Serial Monitor)

Combined long: 0x12345678
Combined long (dec): 305419896

9. Verification

If $x = 0x12345678$, then extracting the bytes should give back the same values:

```
byte rb3 = (x >> 24) & 0xFF; // 0x12  
byte rb2 = (x >> 16) & 0xFF; // 0x34  
byte rb1 = (x >> 8) & 0xFF; // 0x56  
byte rb0 = x & 0xFF; // 0x78
```

10. Key Takeaways

- A long is 32 bits (4 bytes).
- b3 is the most significant byte, b0 is the least significant byte.
- Each byte is shifted to its correct position (24, 16, 8, 0 bits).
- Bitwise OR (|) is used to combine the bytes.
- Use (long) cast to ensure 32-bit shifting.
- This method is useful when receiving multi-byte data (e.g., from sensors, communication, or storage).
- Always verify by extracting the bytes back.

General Form (for any bytes)

```
long x = ((long)b3 << 24) | ((long)b2 << 16) |  
          ((long)b1 << 8) | (long)b0;
```

Place each byte in its correct position, then OR them together.

Forming a long from High and Low Words

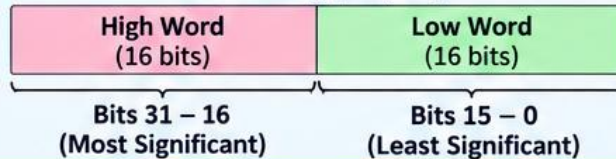
Combine two 16-bit words (high word and low word) to get a 32-bit long value using bitwise operations

Arduino does not have a built-in function for this, so we can create our own `makeLong()` macro.

1. Understanding the Data Size

- In Arduino (AVR), an `int` is 16 bits (2 bytes).
- A `long` is 32 bits (4 bytes).
- A 32-bit `long` can be divided into two 16-bit words:
 - High word** → upper 16 bits (bits 31 – 16)
 - Low word** → lower 16 bits (bits 15 – 0)

32-bit long (4 bytes)



2. `makeLong()` Macro

```
#define makeLong(hi, low) (((long) hi << 16) | (low))
```

What it does?

- Shifts the high 16-bit value 16 positions to the left.
- Combines it with the low 16-bit value using the bitwise OR (`|`) operator.
- The result is a 32-bit long value.

3. How It Works – Step by Step

Suppose: `hi = 258 (0x0102)`
`low = 772 (0x0304)`

Step 1: Shift high word left by 16 bits

`(long)hi << 16` → `0x0102 << 16 = 0x01020000`

After shifting:

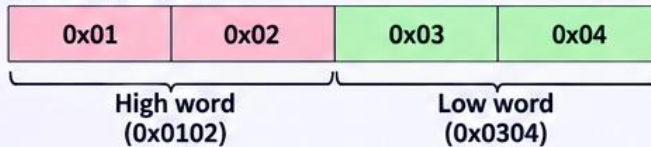


4. Continue – Combine with Low Word

Step 2: Combine using bitwise OR

`0x01020000 | 0x0304 = 0x01020304`

Final 32-bit value:



`0x01020304 = 16909060` (in decimal)

5. Extracting the Words (Reverse Operation)

```
#define highWord(w) ((w) >> 16)  
#define lowWord(w) ((w) & 0xffff)
```

- `highWord(w)` extracts the upper 16 bits.
- `lowWord(w)` extracts the lower 16 bits.

Example:

```
long x = 0x01020304; // 16909060
```

`highWord(x)` `0x0102 = 258` (dec)

`lowWord(x)` `0x0304 = 772` (dec)

6. Complete Example Code (Arduino/C)

```
#define makeLong(hi, low) (((long) hi << 16) | (low))  
#define highWord(w) ((w) >> 16)  
#define lowWord(w) ((w) & 0xffff)
```

```
long longValue = 0x01020304; // 16909060  
  
void setup() {  
  Serial.begin(9600);  
}  
  
void loop() {  
  int hiWord = highWord(longValue);  
  int loWord = lowWord(longValue);  
  Serial.println(longValue); // 16909060  
  Serial.println(hiWord); // 258  
  Serial.println(loWord); // 772  
  longValue = makeLong(hiWord, loWord);  
  Serial.println(longValue); // 16909060  
  delay(1000);  
}
```

Output (Serial Monitor)

16909060
258
772
16909060

7. Summary of Macros

Macro	Function
<code>makeLong(hi, low)</code>	Combines two 16-bit words into a 32-bit long
<code>highWord(w)</code>	Extracts the high 16-bit word (bits 31 – 16)
<code>lowWord(w)</code>	Extracts the low 16-bit word (bits 15 – 0)

8. Key Takeaways

- A `long` is 32 bits (4 bytes) and can be split into two 16-bit words.
- Shift the high word left by 16 bits to place it in the upper half.
- Use bitwise OR (`|`) to combine the words.
- `highWord()` and `lowWord()` can extract the words from a 32-bit value.
- Useful when working with communication protocols, sensors, or storing large values.

9. Important Note

The term word refers to 16 bits when compiling for 8-bit boards such as the Uno; when compiling for 32-bit boards a word is 32 bits and a half word is 16 bits. Although the underlying architecture is different, the preceding code will return high and low 16-bit values on both 8-bit and 32-bit boards.

Questions

- Q1. Predict the output of `int x=7/2; Serial.println(x);` Explain.
- Q2. Explain why `long seconds_per_day=60*60*24` can overflow and write the corrected statement.
- Q3. Differentiate post-increment and pre-increment with one Arduino example.
- Q4. Write an Arduino condition to identify an even integer using `%`.
- Q5. Constrain a sensor value to 100–200 and state the output for 250.
- Q6. For `x=1.9` and `y=-1.9`, distinguish casting to `int` from appropriate rounding functions.
- Q7. Convert 30° to radians and write code to calculate sine; state its approximate value.
- Q8. Differentiate `random(10)` and `random(2,10)`, including output ranges.
- Q9. Write statements to set, read and clear bit 2 of an 8-bit flags variable.
- Q10. For `258=0x0102`, find high/low bytes and write the statement that reconstructs 258.